# HoopSync — Product Brief & Development Plan

## 1. Product Overview

Build a local-first application that captures, parses, aggregates, and displays live and historical NBA 2K game statistics for a group of friends playing together.

The primary goal is to create a shared statistical view of an NBA 2K session where multiple players can connect their own Xbox/PC capture setup and contribute detected game statistics to a common server. The application should transform short-lived NBA 2K UI screens into persistent structured data that can be used for:

- Live game monitoring
- Player stat tracking
- Team/game status
- Opponent history
- Friend-to-friend comparisons
- Historical performance analysis
- Player/build analysis
- Game trends
- Competitive insights
- Future predictive recommendations and gameplans

The initial application should be **local-first**:

```
Xbox
  ↓ HDMI
Elgato HD60 X
  ↓ USB
Local PC
  ↓
Frame Capture / CV / OCR / Parsing
  ↓
Structured Game Events
  ↓
Friends Server
  ↓
Shared Live/Historical Data
  ↓
Each Connected User's Dashboard
```

Version 1 should perform all frame processing locally. The architecture should be designed so processing can eventually be moved to a remote AWS instance if doing so provides meaningful performance, scalability, or maintenance benefits.

---

# 2. Reference UI

The primary full dashboard reference is:

`references/reference_image_main_dashboard_view.png`

The minimalist Live Dashboard reference is:

`references/reference_image_minimalist_view.png`

Use these images as **visual/product references**, not as rigid pixel-perfect specifications.

Note: the mockups' header logo still reads "2K STATS HUB" from an earlier working title. Reskin this to **HoopSync** branding when the dashboard header is implemented — no other change to the reference layout is implied by this.

The visual direction should be:

- Dark
- Modern
- Sports analytics / esports aesthetic
- Dense but readable
- High information density without feeling cluttered
- Rounded cards/panels
- Subtle borders
- Strong typography
- Live status indicators
- Red/blue team differentiation where appropriate
- Responsive desktop layout
- Clear hierarchy between live information and historical information

Do NOT reproduce the native NBA 2K scoreboard UI as the application's primary interface.

Instead, extract the information from NBA 2K's UI and present it through the application's own modern dashboard design.

---

# 3. Primary Product Concepts

The application should revolve around two major systems:

## A. Live Dashboard

Provides the current state of an active game/session.

It should show:

- Current score
- Quarter/game clock
- Shot clock when available
- Team/player statistics
- Player performance
- Opponent statistics
- Recent events
- Connected players
- Capture/processing status
- Current game type
- Current MyPlayer/build
- Live session status

The Live Dashboard should support two modes:

### Full Dashboard

Reference:

`references/reference_image_main_dashboard_view.png`

This should contain:

- Player stat cards
- Modernized live scoreboard
- Team/player stat tables
- Play/event feed
- Top performers
- Game summary
- Session history
- Quick/minimized capture information
- Connected friends
- Elgato/device status

### Minimalist Dashboard

Reference:

`references/reference_image_minimalist_view.png`

This should intentionally remove most secondary information and focus primarily on:

- Score
- Quarter
- Game clock
- Shot clock
- Team status
- Player stat cards
- Connection/live status

The minimalist mode should be suitable for keeping open on a second monitor while actually playing.

---

# 4. Historical Data and CourtIQ Hub

Historical data is a critical part of the application rather than an afterthought.

Expanding on basic stat history, the CourtIQ module acts as the ultimate analytical workspace. It equips users with stats, insights, trends, and interactive graphs to help them digest information and ultimately elevate their gameplay

Every parsed game/event should eventually contribute to a persistent historical dataset.

The system should support analysis such as:

### Individual performance

- Points
- Rebounds
- Assists
- Steals
- Blocks
- FG/FGA
- 3P/3PA
- FT/FTA
- Turnovers

- Plus/minus
- Other statistics exposed by the relevant NBA 2K screen

### Friend comparisons

Examples:

- Most points today
- Highest scoring game this week
- Most assists
- Best FG%
- Best 3P%
- Most wins
- Best performance against a specific opponent
- Best performance using a specific build
- Head-to-head records
- Average stats by game type
- Recent performance trends

### Opponent history

The system should recognize when an opponent has been encountered previously.

For example:

```
Opponent: Player123

Previous Games: 7
Record: 4-3
Avg PTS: 18.4
Avg REB: 5.1
Avg AST: 4.7
Last Played: Yesterday
```

The system should associate opponent identities with historical games whenever identification is sufficiently reliable.

---

# 5. Future Analytics

The underlying data model will support future predictive and qualitative analytical features:

- TruePlayer Rating: A unique algorithm to objectively quantify how good a player actually is beyond standard box-score stats.
- CourtIQ Insights: Performance trends, player strengths/weaknesses, opponent tendencies, build effectiveness, and automated gameplan suggestions.
- Matchup & Predictive Analysis: Head-to-head history and win probabilities based on build matchups.

The above are the high level areas. Below are some more specific items:

- Performance trends
- Player strengths/weaknesses
- Opponent tendencies
- Friend rankings
- Matchup analysis
- Build effectiveness
- Game-type-specific performance
- Suggested gameplans
- Predictive insights
- Recommendations based on historical performance

Do not attempt to implement sophisticated predictive analytics in the first milestone.

Instead, ensure the underlying data model preserves enough information to support these features later.

---

# 6. Data Acquisition Architecture

The primary source of data is an Elgato HD60 X connected to the Xbox and the PC running the application.

The application needs to:

1. Detect the Elgato device.
2. Open the video stream.
3. Continuously acquire frames.
4. Maintain a rolling frame buffer.
5. Detect frames/screens of interest.
6. Extract/crop relevant regions.
7. Parse structured information.
8. Validate the parsed information.

9. Convert results into structured game events.
10. Update the local state.
11. Send relevant structured data to the shared server.

The Elgato capture stream is the core data acquisition mechanism.

---

# 7. Rolling Frame Buffer

A rolling frame buffer should be a core component of Version 1.

This is particularly important because some NBA 2K stat screens may only be visible for approximately 0.5–1 second.

Do NOT design the system as:

```
Detect screen
→ Start recording
```

because the screen may disappear before detection/recording begins.

Instead:

```
Continuous Capture
       ↓
Rolling RAM Buffer
       ↓
Screen Detector
       ↓
Detection Event
       ↓
Capture Pre-Trigger Frames
       +
Capture Current Frames
       +
Capture Short Post-Trigger Window
       ↓
Parse Best Frames
```

For example, maintain approximately 1–2 seconds of recent frames in memory.

When a screen of interest is detected, preserve the preceding frames and continue capturing briefly afterward.

The parser can then select the clearest/highest-confidence frames from the event.

The exact frame rate and buffer duration should be configurable.

---

# 8. Frames of Interest

The application should support multiple classes of structured NBA 2K screens.

Do not assume there will only be one scoreboard format.

Create an extensible detection/parser architecture.

Potential screen/event types include:

```
GAME_SCOREBOARD
PLAYER_STATS
TEAM_STATS
PLAYER_BOX
GAME_LOADING_SCREEN
GAME_RESULT
GAME_START
GAME_END
PLAYER_BUILD_INFO
UNKNOWN
```

The exact taxonomy should be refined after examining real NBA 2K26 captures.

Each frame type should have:

- Detector
- Region-of-interest definition
- Parser
- Validation rules
- Confidence score
- Version/schema identifier

---

# 9. Important State Detectors

The application will likely need a number of lightweight state detectors before reliable parsing can occur.

At minimum, investigate implementation of:

## `is_game_live`

Determine whether the user is actually participating in an active basketball game.

This is necessary to distinguish:

```
Active basketball game
```

from:

```
MyPlayer free roam
Menus
Lobby
Park exploration
Other non-game screens
```

## `game_type`

Identify the current game mode when possible.

Examples:

```
MyPark 5v5
Proving Grounds 2v2
Rec
Pro-Am
Other
Unknown
```

The implementation should be extensible because NBA 2K modes/UI can change.

## `my_player_identity`

Determine which MyPlayer is currently being used.

The user should have a persistent identity:

```
User: Brando
```

but potentially multiple MyPlayer builds:

```
Brando
 ├── Build A
```

```
├── Build B
└── Build C
```

The application should attempt to associate captured statistics with the appropriate build.

If the build cannot reliably be determined during runtime, do NOT invent an identity.

Instead use something such as:

```
myplayer_id = unknown
```

and allow the data to be reconciled later.

---

# 10. Player Stat Box Detection

An important feature is detecting the small player stat/status box that may appear in a corner of the NBA 2K screen.

If present:

```
Detect player box
      ↓
Crop region
      ↓
Parse relevant stats
      ↓
Associate with current user
      ↓
Update live player card
```

This should be treated as a separate detector/parser from the larger scoreboard.

The system should be able to update the user's live stat line even when the full scoreboard is not displayed.

---

# 11. Multi-User Architecture

Multiple friends may be playing in the same game.

Each user may have:

```
Xbox
↓
Elgato
↓
Local application
↓
Local CV/OCR processing
↓
Server
```

The server aggregates structured events.

Example:




The server should not need raw video in Version 1.

Only structured data/events should be transmitted.

This significantly reduces bandwidth and keeps sensitive/raw capture processing local.

---

# 12. Game Session Model

Introduce the concept of a `GameSession`.

A session should have a unique ID.

Conceptually:

```
GameSession
 ├── session_id
 ├── game_type
 ├── started_at
 ├── ended_at
 ├── status
 ├── players
 ├── teams
 ├── score history
 ├── stat events
 ├── detected screens
 └── game result
```

Each local application should associate detected information with the active session.

The server should merge events from multiple users into the same session.

---

# 13. Event-Based Data Model

Do not make the application dependent solely on a continuously changing "current stats" object.

Capture meaningful events.

For example:

```
SCREEN_DETECTED
STAT_UPDATE
PLAYER_BOX_DETECTED
SCORE_UPDATE
GAME_STARTED
GAME_ENDED
PLAYER_IDENTIFIED
GAME_TYPE_IDENTIFIED
```

Each event should ideally include:

```
event_id
session_id
user_id
timestamp
event_type
source
```

```
confidence
payload
```

This will make debugging, synchronization, historical reconstruction, and future analytics significantly easier.

---

# 14. Deduplication

Because the same scoreboard may be visible for multiple frames, the application must avoid generating hundreds of duplicate events.

For example:

```
Frame 101 → scoreboard
Frame 102 → scoreboard
Frame 103 → scoreboard
Frame 104 → scoreboard
...
```

should generally become:

```
One SCOREBOARD_CAPTURE event
```

with multiple frames used internally for parsing/validation.

Implement event debouncing/deduplication.

The exact strategy should be determined during prototyping.

Potential signals:

- Screen-class persistence
- Timestamp
- Parsed stat similarity
- Score similarity
- Player identity
- Session ID

---

# 15. Parsing Strategy

Use a layered approach.

## Layer 1 — Screen Detection

Determine what type of screen/frame is being displayed.

## Layer 2 — Region of Interest

Crop only the relevant portion of the frame.

## Layer 3 — Image Processing

Potential operations:

- Resize
- Grayscale
- Thresholding
- Sharpening
- Contrast adjustment
- Perspective correction if necessary

## Layer 4 — OCR / Structured Recognition

Extract text/numbers.

## Layer 5 — Semantic Parsing

Convert extracted information into application fields.

## Layer 6 — Validation

Check whether the result makes sense.

For example:

```
PTS = 24
REB = 8
AST = 6
```

is plausible.

A parse such as:

```
PTS = 2448
REB = ?
```

```
AST = 6
```

should be flagged for low confidence rather than blindly saved.

---

# 16. Multiple Frames Should Improve OCR

For short-lived screens, don't trust a single frame.

Suppose the scoreboard exists for ~0.5 seconds and produces 15 frames.

The parser should potentially evaluate multiple frames:

```
Frame A → PTS 24
Frame B → PTS 24
Frame C → PTS 24
Frame D → PTS 2?
Frame E → PTS 24
```

The system can use agreement between frames to increase confidence.

This should eventually become a reusable `MultiFrameParser` component.

---

# 17. Training / Data Collection Strategy

Before committing to a machine-learning classifier, determine whether simpler computer-vision methods are sufficient.

This should be an explicit engineering investigation.

## Initial recommendation

Create a small data-collection/testing workflow that allows the developer to capture representative still images from the Elgato stream.

Collect examples of:

- Active gameplay
- Free roam
- Loading screens
- Scoreboard

- Player stat screen
- Team stat screen
- Player stat box visible
- Menus
- Post-game screens
- Different game modes
- Different resolutions/settings if applicable

The most important question is whether static screenshots are sufficient.

If the UI is sufficiently consistent, a conventional detector may be preferable to a trained model.

If the UI varies substantially, create a labeled dataset and train a lightweight classifier/object detector.

Do not prematurely introduce a large vision model.

---

# 18. Developer Test / Verification System

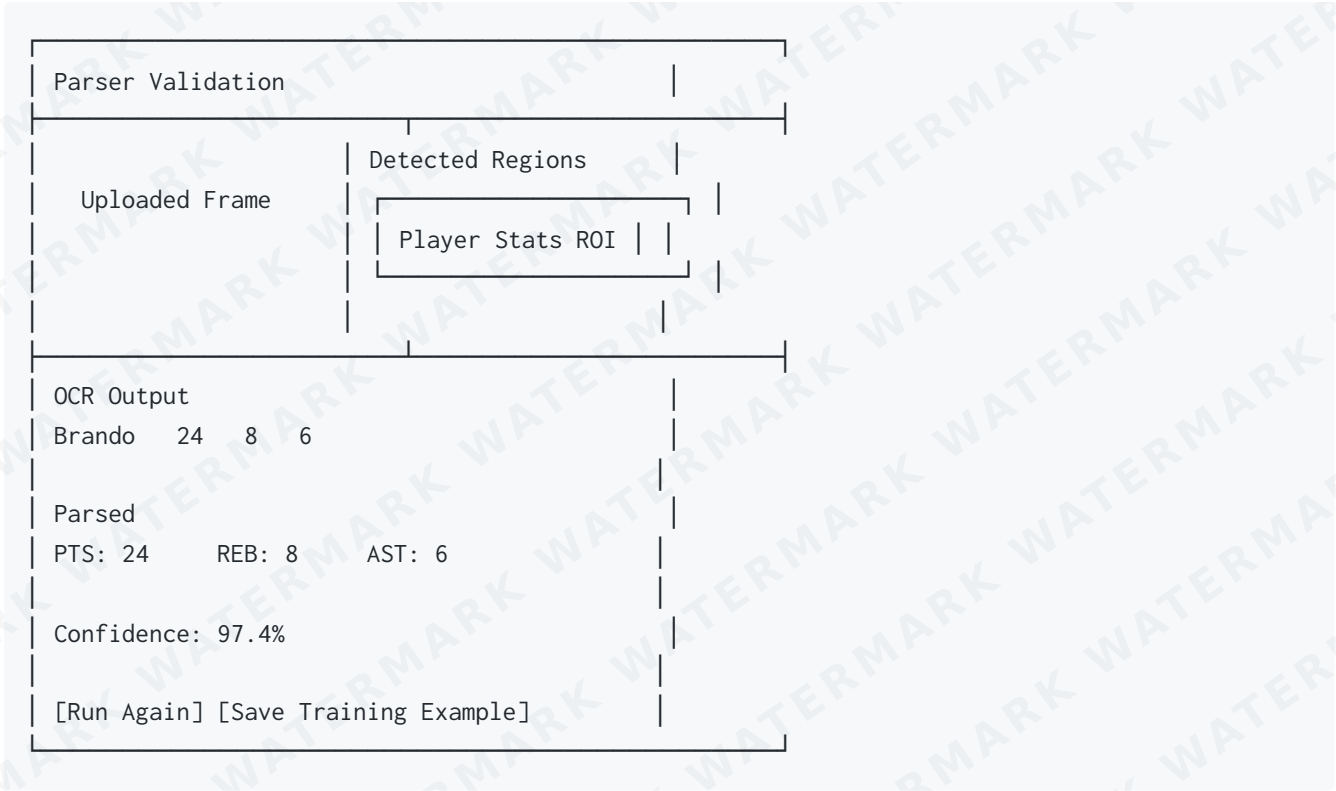
A dedicated development/testing interface should be included.

This is highly recommended because the CV/parser system will be the most failure-prone part of the application.

The developer should be able to:

- Upload a still frame
- Drag/drop an image
- Select a frame from a captured event
- Run a specific detector
- Run a specific parser
- View detected regions
- View OCR output
- View structured parsed output
- View confidence scores
- Compare expected vs actual values
- Save examples to a training/test dataset
- Replay a captured frame sequence
- Run the complete detection pipeline against a sample event

Example:




This can initially be a developer-only route/page rather than a polished end-user feature.

---

# 19. Automated Testing

Create test fixtures from representative screenshots.

The repository should eventually contain something conceptually similar to:

```
tests/
  fixtures/
    scoreboard/
    player_box/
    loading_screen/
    game_result/
    free_roam/
    player_stats/
    team_stats/

  test_detectors/
  test_parsers/
  test_session_state/
  test_deduplication/
  test_event_processing/
```

Tests should verify:

- Correct screen classification
- Correct ROI selection
- OCR parsing
- Numeric extraction
- Player identification
- Game type identification
- Duplicate event suppression
- Session state transitions
- Invalid/low-confidence data rejection

Whenever a real-world parsing failure is discovered, save the problematic frame as a regression fixture.

---

# 20. Server

Version 1 server responsibilities:

- User/session management
- Game session synchronization
- Receiving structured events
- Aggregating player statistics
- Broadcasting live state
- Persisting historical data
- Serving historical queries

The server should NOT initially receive raw video.

Potential architecture:

```
Local Client
    ↓
REST/WebSocket
    ↓
Friends Server
    ↓
Database
    ↓
WebSocket
    ↓
Connected Clients
```

Real-time updates should preferably use WebSockets or another appropriate push mechanism rather than requiring clients to constantly poll.

The specific reusable piece is `/Users/Brando/Projects/forza/apps/friends-server` (and its shared `packages/friends-protocol` types): a small, stateless WebSocket relay with a pure-function, unit-tested session manager (`sessionManager.ts` — join/update/disconnect handling and broadcast-audience computation, no database). Its shape maps directly onto this section's needs — swap Forza's `JOIN` / `UPDATE` /position-payload messages for HoopSync's `GameEvent` messages and `sessionId` stays conceptually equivalent to `GameSession.session_id`. Reuse this pattern (and adapt the code directly) for the event-relay/broadcast layer rather than designing it from scratch. See Section 34a below for what forza does *not* provide (it has no capture/CV/OCR pipeline to reuse) and for a second, more directly applicable reference project for the parsing/validation/dashboard side.

---

# 21. Local Client

The local application is responsible for:

- Elgato detection
- Video capture
- Rolling frame buffer
- Frame classification
- ROI extraction
- OCR/parser execution
- Local validation
- Local event generation
- Communication with the server
- Live dashboard rendering

The application should gracefully handle:

```
Elgato disconnected
Elgato unavailable
Video stream unavailable
Server unavailable
Parser failure
Unknown screen
Low-confidence result
```

The UI should make these states visible.

---

# 22. Device Status

The dashboard should provide a small connection indicator.

Example:

```
Elgato HD60 X
1080p60
● CONNECTED
```

Possible states:

```
CONNECTED
CONNECTING
DISCONNECTED
ERROR
```

This should be a real device status rather than static UI.

---

# 23. Live Dashboard Components

The main dashboard should ultimately include:

## Header

- Application identity
- Live session status
- Game type
- Session timer
- Notifications
- Connected friends
- Current user

## Player Cards

Each connected player gets a compact stat card.

Example:

```
Brando
24 PTS
8 REB
6 AST
● LIVE
```

These should update automatically when new data arrives.

## Modern Live Scoreboard

Use the minimalist scoreboard aesthetic from the reference image rather than reproducing NBA 2K's native scoreboard.

Include:

- Team names
- Scores
- Quarter
- Game clock
- Shot clock
- Fouls
- Timeouts
- Possession indicator

## Team Tables

Display structured player statistics for both teams.

## Live Event Feed

Show recent detected events.

## Top Performers

Rank players in the current game.

## Game Summary

Compare:

- FG%
- 3P%
- Points in paint
- Turnovers
- Other useful team-level metrics

## Session History

Keep this compact on the Live Dashboard.

### Connected Friends

Show who is currently contributing capture data.

---

# 24. Navigation

The application should eventually support the major pages shown in the reference dashboard:

```
LIVE DASHBOARD
GAMES
PLAYERS
LEADERBOARDS
TEAM STATS
COURTIQ
CLIPS
SETTINGS
```

The "Analytics" page described elsewhere in this document is the **CourtIQ** hub (see Section 4/5) — use that name in navigation and UI, not the generic term.

Do not attempt to fully implement every page in the first milestone.

Recommended development order:

1. Live Dashboard
2. Game/session history
3. Players
4. Leaderboards
5. Team statistics
6. CourtIQ
7. Clips
8. Settings/developer tooling refinement

---

# 25. Clips / Captures

Recent captures should NOT dominate the Live Dashboard.

The reference UI should treat them as a small secondary component.

Raw frame sequences should primarily be useful for:

- Debugging
- Parser verification
- Event auditing
- Optional clip generation

Avoid storing every captured frame permanently.

Use the rolling buffer and event-based capture architecture to minimize disk usage.

---

# 26. Database Design

Design the database around normalized entities and historical events.

Likely entities include:

```
User
Player
MyPlayerBuild
GameSession
GameParticipant
Team
Opponent
GameEvent
StatSnapshot
GameResult
GameType
DetectedScreen
CaptureEvent
```

The exact schema should be determined during implementation planning.

Important distinction:

A real person/user should be different from a MyPlayer/build.

Example:

```
User
  Brando
     │
     ├── MyPlayer Build A
     ├── MyPlayer Build B
     └── MyPlayer Build C
```

This allows aggregate Brando statistics while also supporting build-specific analysis.

---

# 27. Identity Resolution

Identity detection will not always be perfect.

The system should use confidence-based identification.

For example:

```
Player detected:
Brando

Confidence:
0.98
```

or:

```
Player detected:
Unknown

Confidence:
0.42
```

Low-confidence identities should not silently become permanent incorrect historical records.

The application should support later correction/reconciliation.

---

# 28. Historical Opponent Resolution

Opponent matching should eventually support:

```
Exact known player
Potential match
Unknown player
```

If an opponent has previously been encountered, surface that information during the current game.

Example:

```
OPPONENT DETECTED

IsoKing

Previous Games: 7
Record: 4-3
Avg PTS: 18.4
Avg AST: 4.7
Last Played: Aug 17
```

This becomes a foundation for future matchup analytics.

---

# 29. Performance Requirements

The local detection pipeline should be designed for low latency.

Target behavior:

```
HDMI frame
   ↓
Capture
   ↓
Detection
   ↓
Parsing
   ↓
Event
   ↓
Server
   ↓
Dashboard
```

should feel close to real-time.

Do not sacrifice the entire application's performance by running an expensive model against every frame.

Prefer:

```
Cheap detector
      ↓
Potential screen
      ↓
Expensive parser only when necessary
```

This is particularly important for a continuously running capture application.

---

# 30. Processing Pipeline

A recommended architecture is:

```
ElgatoCapture
      ↓
FrameBuffer
      ↓
FramePreprocessor
      ↓
ScreenDetector
      ↓
EventDetector
      ↓
ROIExtractor
      ↓
Parser/OCR
      ↓
MultiFrameAggregator
      ↓
Validation
      ↓
GameStateManager
      ↓
EventStore
      ↓
ServerSync
      ↓
Live Dashboard
```

Each component should have a clear interface so that detectors/parsers can be replaced independently.

---

# 31. Development Phases

## Phase 0 — Repository/Architecture Investigation

**Status: complete.** See Section 34a for full findings. Summary:

1. `/Users/Brando/Projects/forza` was inspected. It does **not** have an Elgato/capture-card SDK integration, a CV pipeline, or OCR — its "Elgato" support (`apps/web/src/widgets/CaptureFeedPanel.tsx`) is a browser `getUserMedia()` panel that lists whatever UVC-style video devices are available, which happens to include Elgato output when Elgato's own software exposes it that way. There is no frame buffer, no screen detector, no parser.
2. forza's genuinely reusable asset is `apps/friends-server` + `packages/friends-protocol`: a stateless, unit-tested WebSocket relay (see Section 20). Reuse/adapt this directly for HoopSync's event-relay layer.
3. `/Users/Brando/Projects/GamedayBetTrackerr` (not originally listed in this brief, discovered during Phase 0) turned out to be the more relevant reference project overall — it already solves the "turn a screenshot into structured, confidence-scored, human-correctable data" problem end-to-end. See Section 34a for what to borrow from it.
4. Data models/utilities: forza's `packages/telemetry` (packet parsing, pure functions, fixture-based tests) is a reasonable testing-philosophy reference but not directly reusable (different domain). GamedayBetTrackerr's Prisma schema conventions (Section 34a) are the more relevant data-model precedent for HoopSync.
5. Technologies: forza uses pnpm workspaces, Vite/React, and a raw Node WebSocket/HTTP server. GamedayBetTrackerr uses Next.js (App Router), TypeScript, Prisma/SQLite, Tailwind. See Section 34a for the recommended stack split.

---

## Phase 1 — Elgato Capture Prototype

Goal:

```
Elgato → Application → Live frames
```

Decided architecture (see Section 34a): an **Electron desktop app**, cross-platform (Windows + Mac), using Chromium's `getUserMedia()`/Canvas APIs in the renderer — the same mechanism forza's `CaptureFeedPanel.tsx` already proves works against Elgato output — rather than native DirectShow/AVFoundation code per OS. Electron gets the "always-on installable app" UX the brief wants (Section 21/22) without writing two native capture backends.

Implement:

- Device discovery (enumerate video-input devices, persist the chosen one, matching forza's pattern)
- Stream initialization
- Frame acquisition (Canvas frame grabs from the `<video>` element)
- FPS monitoring
- Resolution detection

- Rolling buffer (in the Electron main or renderer process — see Section 7)
- Frame preview
- Capture diagnostics
- Device status surfaced as CONNECTED/CONNECTING/DISCONNECTED/ERROR (Section 22)

Success criterion:

The application can reliably display/save frames from the HD60 X for extended periods, on both a Windows and a Mac test machine.

---

## Phase 2 — Frame Dataset / Developer Tooling

Build the initial frame validation tool.

Allow the developer to:

- Capture stills
- Upload stills
- Organize examples by screen type
- View frames
- Run detectors
- Inspect ROIs
- Run OCR
- Inspect parsed output

Use real NBA 2K26 frames.

Determine whether static screenshots are sufficient or whether a controlled real-time capture session is necessary.

**Reuse `/Users/Brando/Projects/GamedayBetTrackerr`'s ingestion UI as a direct template**: `UploadDropzone.tsx`, `SlipReviewForm.tsx`, and `LegEditorRow.tsx` already implement "upload an image → see extracted structured fields → correct them inline" — exactly the interaction this developer tool needs, just aimed at NBA 2K screens instead of parlay slips. Adapt rather than design from scratch.

---

## Phase 3 — Screen Detection

Implement initial detectors.

Start with:

1. Active game
2. Scoreboard
3. Player stat box
4. Loading/game information screen

Do not attempt every possible NBA 2K screen immediately.

Measure detection accuracy.

---

## Phase 4 — Scoreboard Parser

Build the first complete end-to-end parser:

```
Elgato
→ frame
→ scoreboard detected
→ rolling buffer event
→ ROI
→ OCR
→ parsed stats
→ validation
→ structured event
```

This is the first major milestone.

**Decided approach (see Section 34a): start with deterministic OCR, not a vision model.** Build the parser as a `ScreenParser` interface (mirroring GamedayBetTrackerr's `VisionParser` interface) with an initial deterministic implementation (Tesseract-based, mirroring `OfflineOcrParser.ts` / `ocrEngine.ts`), a pure `confidenceGate`-style validation function (mirroring `confidenceGate.ts`), and Zod-validated output schemas carrying a per-field `confidence`, mirroring `ParsedSlipDraftSchema`. This keeps the door open to add a Claude-vision or trained-classifier implementation later as a second class behind the same interface, without touching call sites — validating this empirically (Section 17/34) stays possible even though deterministic OCR is the starting point.

---

## Phase 5 — Live Game State

Implement:

- GameSession

- Players
- Teams
- Game type
- Score
- Quarter
- Clock
- Player statistics
- Event processing

---

## Phase 6 — Server Synchronization

Connect multiple local clients.

Test:

```
Client A → server
Client B → server
Client C → server
Client D → server
```

with each client contributing partial information.

The server should merge the data into one shared GameSession.

---

## Phase 7 — Main Live Dashboard

Implement the dashboard matching:

`references/reference_image_main_dashboard_view.png`

Prioritize:

1. Live scoreboard
2. Player cards
3. Team/player statistics
4. Live event feed
5. Connected friends
6. Device status

Secondary components can follow.

---

## Phase 8 — Minimalist Dashboard

Implement:

`references/reference_image_minimalist_view.png`

Allow users to switch between:

```
FULL DASHBOARD
MINIMAL VIEW
```

The minimalist view should be optimized for a second monitor.

---

## Phase 9 — Historical Data

Implement:

- Game history
- Player history
- Opponent history
- Friend comparisons
- Leaderboards
- Aggregated statistics

---

## Phase 10 — Analytics

Once sufficient historical data exists, implement:

- Trends
- Performance comparisons
- Build comparisons
- Opponent tendencies
- Matchup history
- Game-type performance
- Competitive insights

Predictive features should come after sufficient data quality has been established.

---

# 32. Important Engineering Principles

## Data quality over feature quantity

The application will only be as valuable as its parsed historical data.

A wrong stat silently stored in the database is worse than a missing stat.

Favor:

```
Unknown / low confidence
```

over:

```
Incorrect but apparently valid
```

## Local-first

Keep video and expensive processing local in Version 1.

Transmit structured data rather than raw video.

## Modular detection

Every screen detector should be independently testable.

## Modular parsing

Every parser should have its own fixtures/tests.

## Event-driven architecture

Use events rather than relying entirely on mutable state.

## Configuration-driven ROIs

Do not hard-code every coordinate throughout the application.

Screen/ROI definitions should be centralized and configurable.

## Observability

The developer should be able to answer:

> Why did the application think this was a scoreboard?

and:

> Why did it parse this player's points as 28 instead of 23?

This is why the validation/debugging interface is important.

---

# 33. Initial Deliverable

The first working milestone should NOT attempt to finish the entire product.

The first meaningful end-to-end demonstration should be:

```
Xbox
 ↓
Elgato HD60 X
 ↓
Local application
 ↓
Rolling frame buffer
 ↓
Scoreboard detector
 ↓
Scoreboard parser
 ↓
Validated structured stats
 ↓
Server
 ↓
Live Dashboard
```

with one user first.

Once that works reliably, expand to:

```
Multiple players
 ↓
Shared GameSession
 ↓
Aggregated live dashboard
 ↓
Persistent historical database
```

This validates the fundamental technical premise before significant time is spent on secondary UI/features.

---

# 34. Claude Code Instructions

Before implementing the application:

1. Inspect the existing repository structure.
2. Inspect `/Users/Brando/Projects/forza`.
3. Identify reusable Elgato and friends-server code.
4. Inspect the reference images.
5. Produce a proposed architecture.
6. Identify uncertainties and technical risks.
7. Determine the smallest viable Phase 1 implementation.
8. Establish tests before adding complex CV/ML functionality where practical.
9. Do not prematurely implement ML if deterministic CV/OCR is sufficient.
10. Keep components modular enough that detectors/parsers can later be replaced with trained models.
11. Keep the local processing/server boundary explicit.
12. Prefer an incremental vertical-slice approach over attempting to build the entire application at once.

The most important technical question to answer early is:

> **Can we reliably detect and parse the short-lived NBA 2K screens using deterministic computer vision/OCR, or do we need a trained screen classifier/object detector?**

Build the development tooling so this question can be answered empirically using real Elgato captures.

The application should ultimately feel like a **live sports analytics platform built around NBA 2K**, not simply an OCR utility.

The Elgato/capture system is the data acquisition mechanism; the dashboard, historical database, analytics, and competitive comparison system are the actual product.

---

# 34a. Decisions & Additions (Phase 0 addendum)

This section documents the Phase 0 investigation findings and the architecture decisions made from them, so this brief stays the source of truth rather than requiring re-derivation from scratch each session. Written after inspecting both `/Users/Brando/Projects/forza` and `/Users/Brando/Projects/GamedayBetTrackerr` and reviewing the decisions with Brando.

## Repository investigation findings

**forza does not have an Elgato/capture-card pipeline to reuse.** Its only "Elgato" reference is `apps/web/src/widgets/CaptureFeedPanel.tsx`, a browser panel that calls `getUserMedia()` / `enumerateDevices()` against whatever UVC-style video input is available — which includes Elgato output once Elgato's own capture software exposes it that way, but there is no frame buffer, screen detector, ROI system, OCR, or parser behind it. Forza's actual data source is UDP telemetry the game broadcasts natively — a completely different acquisition mechanism from HDMI capture + computer vision. Don't expect to find CV/OCR code there.

**forza's genuinely reusable piece is** `apps/friends-server` **+** `packages/friends-protocol`: a small, stateless WebSocket relay with a pure-function, fully unit-tested session manager (join/update/disconnect, broadcast-audience computation, no database — sessions live and die in memory). This is a strong architectural template for HoopSync's event-relay layer (Section 20/11) — adapt it directly, swapping Forza's position-update messages for `GameEvent` messages and keeping `sessionId` ≈ `GameSession.session_id`.

`/Users/Brando/Projects/GamedayBetTrackerr` **(aka "SweatLine") is the more relevant reference overall, though not mentioned in the original brief.** It's a friend-group dashboard that ingests a screenshot (a parlay betting slip), runs it through a vision/OCR pipeline, produces confidence-scored structured data, lets the user correct low-confidence extractions, and displays everyone's data on a shared dark/neon live dashboard with a leaderboard — the same shape as HoopSync's "screen → structured event → shared dashboard" pipeline, just for a different domain. Specific pieces worth reusing directly:

- `src/lib/providers/vision/` — a `VisionParser` interface (`parseSlipImage(image): Promise<ParsedSlipDraft>`) with three interchangeable implementations: `MockVisionParser`, `OfflineOcrParser` (Tesseract-based, in `offline/ocrEngine.ts` + `offline/extractor.ts`), and `ClaudeVisionParser`. This is exactly the "cheap deterministic detector vs. trained/ML parser, swappable behind one interface" architecture Section 34 asks for (point 9-10) — don't design this from scratch, adapt this one.
- `src/lib/providers/vision/offline/confidenceGate.ts` — a pure function that takes a parsed draft and returns `{ accepted, reasons }`, checking both "were all expected fields found" and "was the average per-field confidence high enough." Direct template for Section 15 Layer 6 (Validation) and Section 27 (confidence-based identity resolution).
- `src/lib/types/vision.ts` — Zod schemas (`ParsedSlipDraftSchema`) where every extracted sub-object carries its own `confidence: number` field. Template for HoopSync's

`GameEvent` / `StatSnapshot` payload validation (Section 13).

- **Ingestion review UI** (`UploadDropzone.tsx`, `SlipReviewForm.tsx`, `LegEditorRow.tsx`) — upload → see extracted fields → correct inline → confirm. Direct template for Section 18's developer verification tool; the pattern also generalizes well if HoopSync ever wants an end-user-facing "confirm what we detected" step for low-confidence events, not just a dev-only tool.
- `prisma/schema.prisma` **conventions** — SQLite via Prisma, with categorical fields kept as plain `String` (validated by a Zod/TS union in application code) rather than native Prisma `enum`, specifically so a later migration to Postgres needs no schema rewrite; per-record `confidence` / `needsReview` columns. Good template for Section 26.
- `src/lib/sweat/sweatEngine.ts` — a pure, independently unit-tested scoring module, isolated from UI and API routes. This is the right shape for **TruePlayer Rating** (Section 5) once there's enough historical data to compute it: a standalone, pure, heavily-tested module that consumes normalized historical stats and produces a rating, not logic embedded in a dashboard component or API handler.

## Decisions made

**Platform: Windows and Mac.** The capture app must run on whichever machine each friend's Elgato is plugged into, which won't be uniform across the group.

**Capture app shell: Electron.** Chromium's `getUserMedia()` /Canvas APIs (the same mechanism forza's `CaptureFeedPanel` already proves works against Elgato output) run identically on both platforms, so Electron gets cross-platform capture "for free" without writing separate DirectShow (Windows) and AVFoundation (Mac) native integrations. Electron also gets the always-on, installable-app UX Section 21/22 wants, versus a browser tab someone has to remember to keep open. Revisit only if `getUserMedia` frame quality/latency proves insufficient in Phase 1 — the brief's original "native service via ffmpeg/OpenCV" option remains the fallback.

**OCR approach: deterministic first.** Start Phase 3/4 with a Tesseract-based deterministic pipeline (per Section 17's own stated preference against premature ML), built behind a `ScreenParser` interface modeled on GamedayBetTrackerr's `VisionParser`. This preserves the ability to empirically test a Claude-vision or trained-classifier implementation later (Section 34's central open question) as a second implementation of the same interface — a config swap, not a rewrite — once real capture data shows whether deterministic OCR is accurate enough on NBA 2K's UI.

**Recommended stack split:**

| Component | Stack | Rationale |
| --- | --- | --- |
| Capture app (Elgato → frames → local parsing) | Electron + TypeScript + React | Cross-platform capture via `getUserMedia`; reuses forza's proven `CaptureFeedPanel` pattern |
| Event-relay/friends server | Adapted from forza's `apps/friends-server` (Node + `ws`) | Already built, tested, stateless — matches Section 20's needs almost exactly |
| Dashboard + historical data + REST API | Next.js (App Router) + TypeScript + Prisma/SQLite + Tailwind | Matches GamedayBetTrackerr's proven shape for "ingest → review → store → shared live dashboard"; Prisma/SQLite keeps a later Postgres migration cheap (Section 26) |

The event-relay server and the Next.js dashboard/API can start as two small services (as forza and GamedayBetTrackerr each are today) and be merged or kept separate depending on how Phase 6 (Server Synchronization) goes — no need to decide that permanently now.

**Repository status:** HoopSync is not yet a git repository and has no code — this session's deliverable was this brief update only. Bootstrapping the repo (git init, scaffolding both the Electron capture app and the Next.js server/dashboard per the stack above) is the natural next session's work, starting from Phase 1.

## TruePlayer Rating and CourtIQ — schema implication

Both concepts already appear in Sections 4-5 above (CourtIQ as the historical analytics hub, TruePlayer Rating as the proprietary player-quality metric) and now also in navigation (Section 24). Per Section 5's existing guidance, do not attempt to compute TruePlayer Rating in the first milestone — but Section 26's database design should explicitly reserve room for its likely future inputs so the historical data doesn't need backfilling later: per-game advanced stats (plus/minus, FG/3P/FT efficiency already listed), opponent strength/record at time of game, MyPlayer build metadata, and game-type context. This is a specific instance of Section 26's existing "preserves enough information" principle, called out here because TruePlayer Rating is now a named, committed feature rather than a placeholder idea.